

Signal Propagation in Single Cycle RV32I

Students: Kevin Gomez, Karellen Velazquez

Class: CHIC 4082

Professor: Prof. Juan Patarroyo

1. Introduction

This report documents the testing and validation of a 32-bit RISC-V processor implemented in [CircuitVerse](#). The goal is to verify the major processor components and confirm correct instruction execution across the datapath.

The project history is available in [our GitHub repository](#).

The document is organized into three parts:

1. **Part 1 - Instruction Memory Decoding:** Identification and analysis of the instruction set stored in the processor's instruction memory
2. **Part 2 - Individual Component Testing:** Verification of each processor subsystem in isolation
3. **Part 3 - Processor Integration Testing:** End-to-end validation of the complete processor executing the full instruction sequence

Each part includes test protocols, measurements at critical signal points using the CircuitVerse Flag element, and verification that each component or instruction meets its functional specification.

The processor operates on 8-bit data but uses 32-bit instruction and program counter (PC) values, as specified in the reference design. Flags are placed throughout the design to support signal observation and verification during simulation.

2. Processor Architecture Overview

The RISC-V processor implemented in this project follows a modular design with distinct functional blocks that interact through well-defined signal interfaces. Understanding the architecture is necessary for designing test strategies and interpreting component behavior in context.

2.1. System Architecture

The processor consists of the following primary subsystems:

- **Instruction Memory (instr_mem):** Holds the program instructions to be executed. Stores 8 words of 32-bit instruction data.
- **Register File (reg_file):** Provides 8 registers for operand storage and intermediate computation results.

- **Control Unit (control_unit):** Decodes instructions and generates control signals that coordinate subsystem operation.
- **Immediate Generator (imm_gen):** Extracts and extends immediate values embedded within instructions.
- **Arithmetic Logic Unit (ALU):** Performs arithmetic and logical operations on operands.
- **Data Memory (data_mem):** Stores and retrieves program data during execution.

These components operate synchronously, with state changes occurring on clock edges. The processor uses synchronous reset, so the reset signal must be asserted high and followed by a clock pulse to reinitialize the PC and memory elements.

2.2. Datapath and Signal Flow

Instructions are fetched sequentially from instruction memory based on the current PC value. The control unit decodes each instruction, extracting opcode, function codes, and register indices. Operands are retrieved from the register file, immediate values are generated as needed, and the ALU or data memory performs the specified operation. Results are written back to the register file, and the PC advances to the next instruction address.

Throughout execution, key signal points such as instruction values, operand data, ALU results, and memory access addresses are available for measurement and verification, enabling direct observation of processor operation at any instruction cycle.

3. Part 1: Instruction Memory Decoding

The instruction memory block contains the program instructions used during validation. Which means we must first decode each one to understand the sequence executed by the processor during testing.

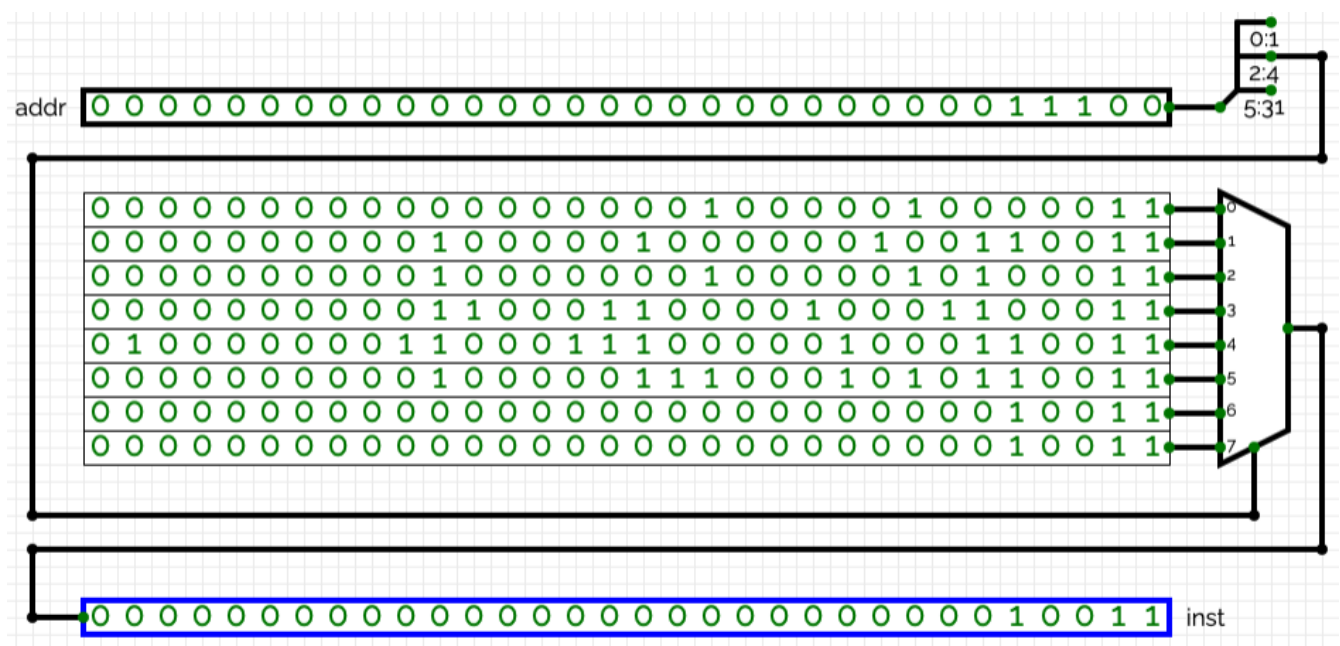


Figure 1. Instruction Memory Circuit Diagram

3.1. Decoding Protocol

To decode the instruction memory contents, we accessed the `instr_mem` block in the CircuitVerse design and observed the binary instruction values stored at each memory address. Each 32-bit instruction follows the RISC-V encoding format, with specific bit fields defining the operation, source registers, destination register, and immediate values.

The decoding process involved:

1. Identifying the opcode (bits [6:0]) to determine instruction type
2. Extracting the rd (destination register), rs1 (source 1), and rs2 (source 2) fields
3. Parsing function code fields (funct3, funct7) that refine the operation
4. Decoding immediate values for I-type, S-type, and B-type instructions
5. Translating the binary encoding to RISC-V assembly code

3.2. Documented Instructions

The following table shows the decoded instruction memory contents. The instruction memory stores eight 32-bit instructions, each located at word-aligned addresses from `0x00` to `0x1C`.

Table 1. Instructions

PC	Binary Instruction	Hex	Format	Assembly Instruction
0x00	000000000000000010000010000011	0x00002083	I-type	lw x1, 0(x0)
0x04	0000000001000001000000100110011	0x00208133	R-type	add x2, x1, x2
0x08	0000000001000000010000010100011	0x002020A3	S-type	sw x2, 1(x0)
0x0C	0000000001100011000010001100011	0x00318463	B-type	beq x3, x3, 8
0x10	01000000011000111000001000110011	0x40638233	R-type	sub x4, x7, x6
0x14	0000000001000001110001010110011	0x0020E2B3	R-type	or x5, x1, x2
0x18	00000000000000000000000000010011	0x00000013	I-type	addi x0, x0, 0 (nop)
0x1C	00000000000000000000000000010011	0x00000013	I-type	addi x0, x0, 0 (nop)

The instruction sequence executed by the processor is:

```
0x00: lw    x1, 0(x0)
0x04: add   x2, x1, x2
0x08: sw    x2, 1(x0)
0x0C: beq   x3, x3, 8
0x14: or    x5, x1, x2
```

```
0x18: addi x0, x0, 0
0x1C: addi x0, x0, 0
```

The branch instruction at address `0x0C` compares `x3` with itself, so the branch condition is true. With an immediate value of `8`, the branch target is `0x14`, which skips the instruction at `0x10` when the branch is taken.

The final two instructions are `addi x0, x0, 0`, which is the RISC-V no-operation instruction (`nop`). Since `x0` is always zero, these instructions do not modify the processor state.

These instructions form the test sequence used during Part 3 processor validation.

4. Part 2: Individual Component Testing

Component-level testing validated that each processor subsystem functions correctly in isolation before integration into the complete datapath.

The testing methodology for each combinational component involved:

1. Developing a test suite using the CircuitVerse TESTBENCH that exercises various operational modes and edge cases
2. Observing test results and ensuring they all pass
3. Documenting results to provide evidence of correct operation
4. Exporting test cases as CSV files for future use and documentation

NOTE

Some input bits are marked with "X". These are don't-care bits, so injecting either "0" or "1" at the corresponding position yielded the same result for that test case. Also, sequential components sometimes involved applying inputs and then pulsing the clock to observe changes in the outputs and at times asserting the reset signal to verify proper initialization behavior. This is noted in the test case descriptions where applicable. Finally screenshots of the circuits displaying the correct results were only shown in some cases and not all to avoid overloading the document with images, but all test cases shown in tables were verified to pass in CircuitVerse.

The following subsections document the testing of each major component:

4.1. Instruction Memory

The Instruction Memory is a read-only combinational memory that provides a 32-bit instruction word for a given 32-bit address. It supplies the next instruction to the instruction fetch stage based on the program counter.

4.1.1. Specifications

The figure below shows the Instruction Memory circuit in CircuitVerse with the input and output signals labeled.

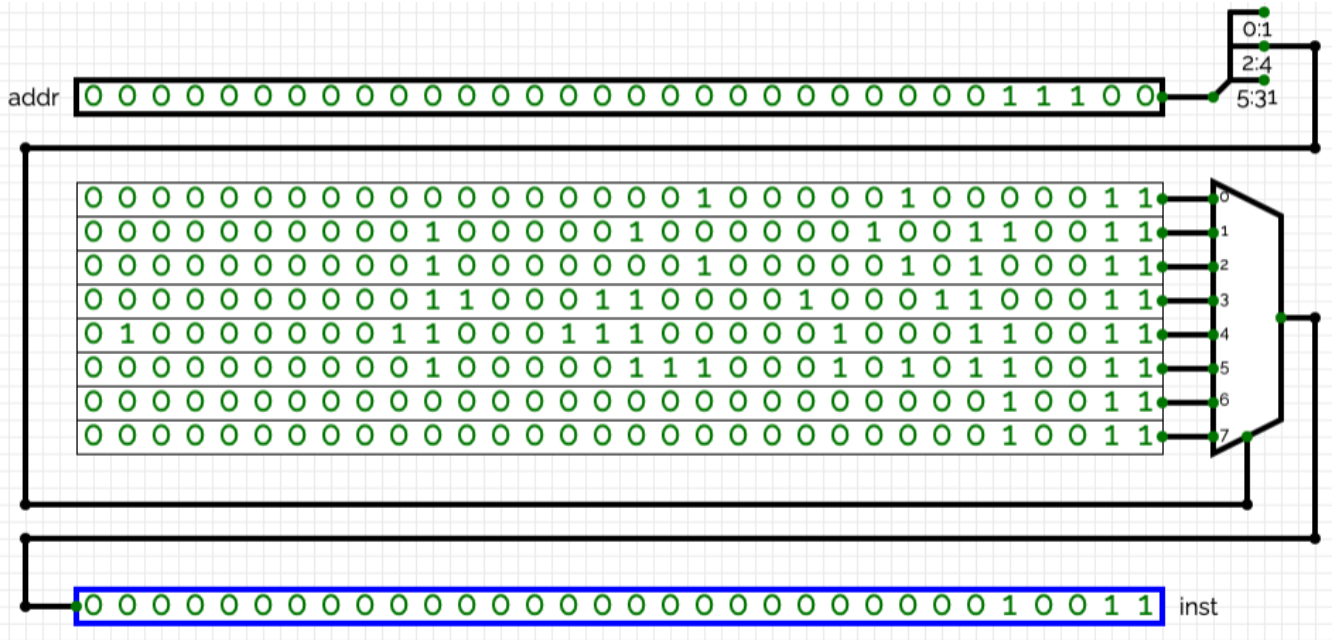


Figure 2. Instruction Memory Circuit Diagram

- **addr** (32-bit): Byte address input (word-aligned expected).
- **inst** (32-bit): Instruction word output stored at the given address.

The component performs a direct address-to-word lookup. Addresses are expected to be aligned to 4-byte boundaries.

4.1.2. Testing Methodology

Verification applies known 32-bit addresses and checks that the component returns the expected 32-bit instruction stored at each location. The tests cover the full range of addresses used during validation, ensuring correct instruction retrieval for all program instructions.

4.1.3. Test Cases and Results

Table 2. Instruction Memory Test Cases

Input addr	Output inst
00000000000000000000000000000000 (0)	00000000000000000000000000000000
00000000000000000000000000000100 (4)	00000000000000000000000000000000
00000000000000000000000000000100 (8)	00000000000000000000000000000000
000000000000000000000000000001100 (12)	00000000000000000000000000000000
000000000000000000000000000001000 (16)	01000000000000000000000000000000
0000000000000000000000000000010100 (20)	00000000000000000000000000000000
0000000000000000000000000000011000 (24)	00000000000000000000000000000000
0000000000000000000000000000011100 (28)	00000000000000000000000000000000

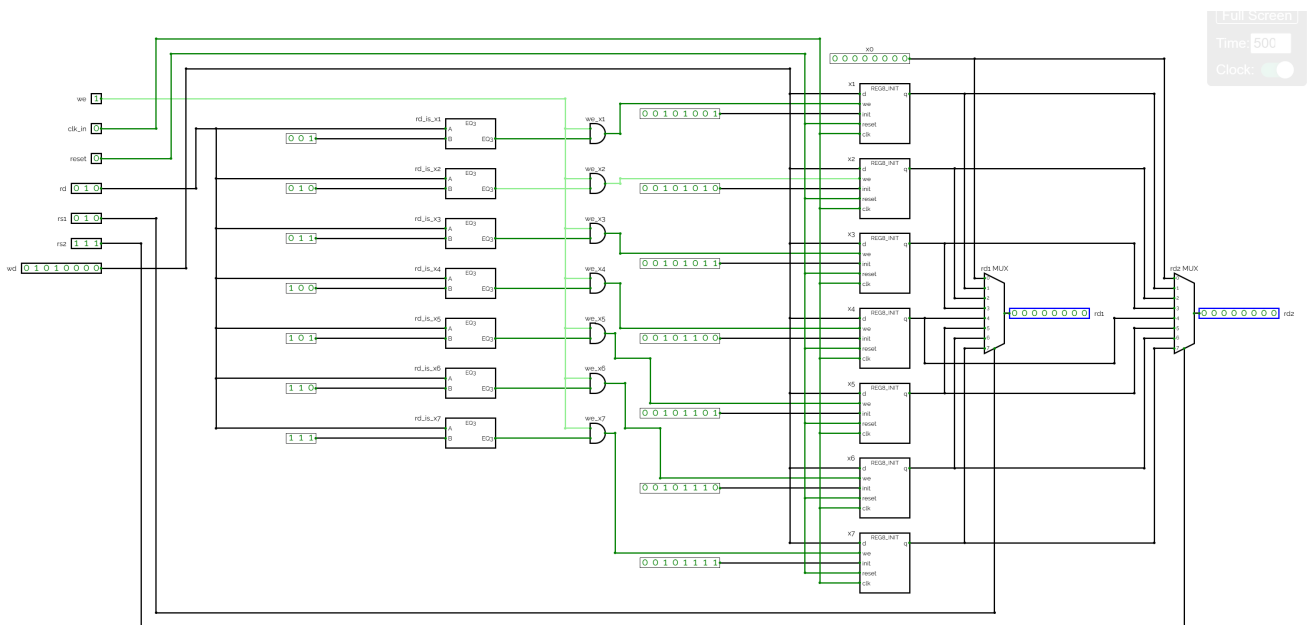
4.1.4. Verification

All supplied addresses returned the expected 32-bit instruction words. The tested address range confirms correct lookup and address decoding.

4.2. Register File

The Register File is a sequential component used to store and provide access to the processor registers. In this simplified RV32I single-cycle processor, the register file contains eight 8-bit registers. The component has two read ports and one write port, allowing the processor to read two source operands and write one destination value.

4.2.1. Specifications



The Register File component has the following input and output signals:

Table 3. Register File Signals

Signal	Bitwidth	Description
we	1	Write enable signal. When we = 1 , the selected destination register is updated on a clock pulse.
clk_in	1	Clock input. Writes and reset behavior occur synchronously when the clock is triggered.
reset	1	Synchronous reset signal. When enabled and the clock is triggered, the register file returns to its initialized values.
rd	3	Destination register address used for writing.
rs1	3	Source register address for the first read port.
rs2	3	Source register address for the second read port.
wd	8	Write data input. This is the value stored into the selected register when we = 1 .

Signal	Bitwidth	Description
rd1	8	First read data output, selected by rs1.
rd2	8	Second read data output, selected by rs2.

The read ports are combinational, so **rd1** and **rd2** change when **rs1** and **rs2** change. The write operation is synchronous, so the selected register changes only after a clock pulse when **we = 1**.

4.2.2. Testing Methodology

The Register File was tested inside CircuitVerse by manually changing the input values and observing the outputs **rd1** and **rd2**.

The tests were designed to verify the following behavior:

1. The register file responds correctly to reset.
2. The read ports correctly output the values selected by **rs1** and **rs2**.
3. A register can be written when **we = 1** and the clock is triggered.
4. A register is not modified when **we = 0**, even if a clock pulse is applied.
5. The register file can return to its initialized state after reset.

The complete test cases are documented in [tests/Register File Tests.csv](#).

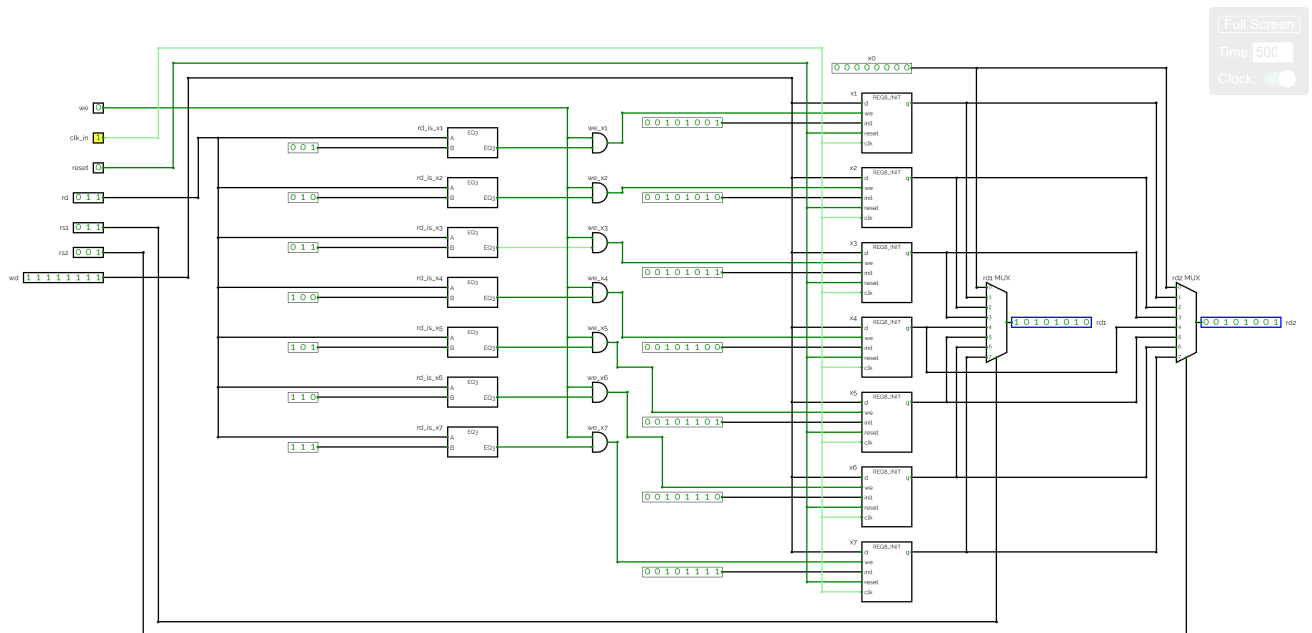
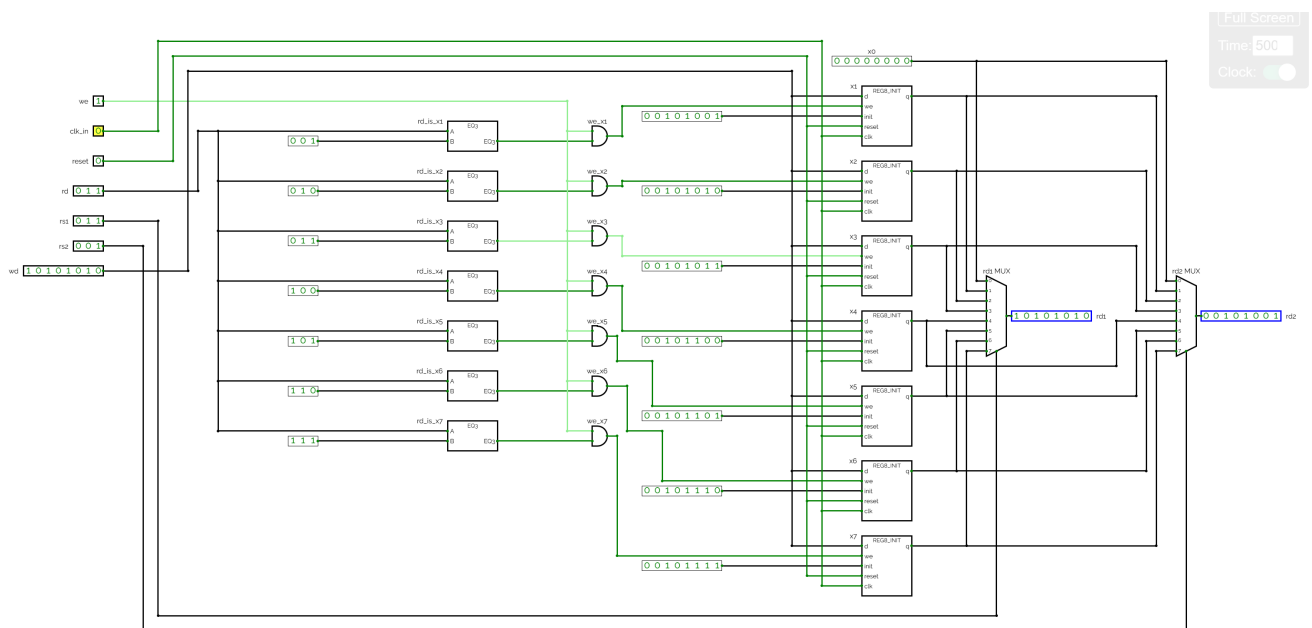
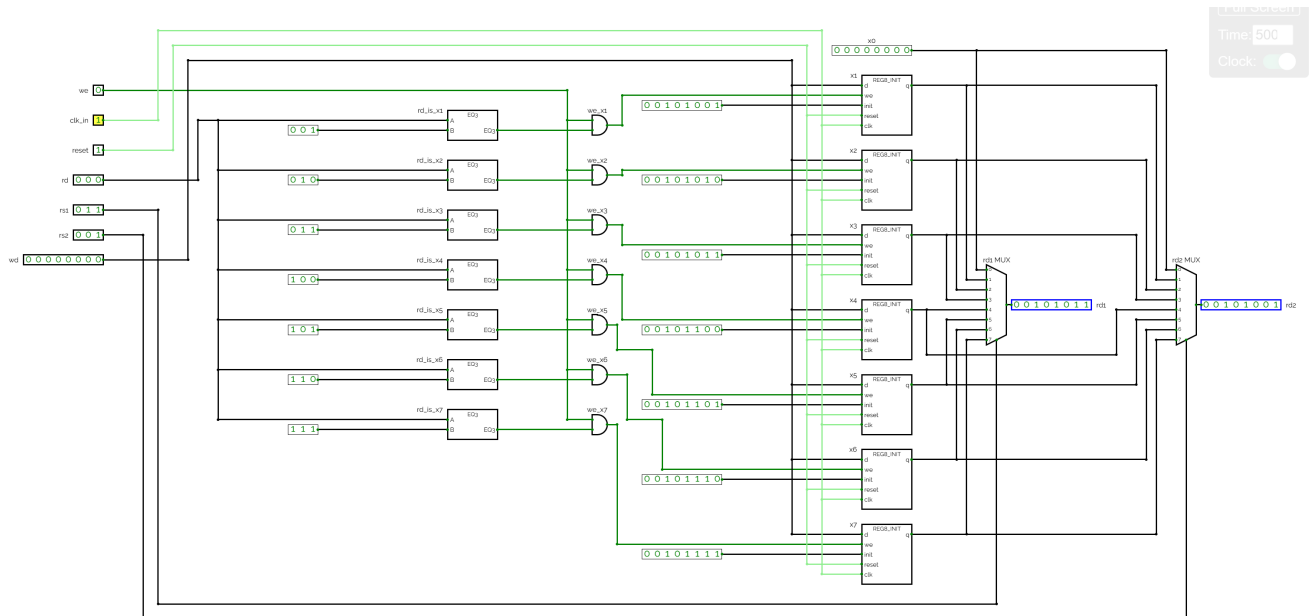
4.2.3. Test Cases and Results

Table 4. Register File Test Cases

Test	Inputs Applied	Expected Output	Result
RF_RESET_X3_X1	we=0, reset=1, rd=000, rs1=011, rs2=001, wd=00000000, clock pulse	rd1=00101011, rd2=00101001	Passed
RF_WRITE_X3_AA	we=1, reset=0, rd=011, rs1=011, rs2=001, wd=10101010, clock pulse	rd1=10101010, rd2=00101001	Passed
RF_BLOCK_WRITE_X3	we=0, reset=0, rd=011, rs1=011, rs2=001, wd=11111111, clock pulse	rd1=10101010, rd2=00101001	Passed
RF_RESET_AFTER_WRITES	we=0, reset=1, rd=000, rs1=011, rs2=001, wd=00000000, clock pulse	rd1=00101011, rd2=00101001	Passed

4.2.4. Evidence

The following images show the internal CircuitVerse tests used to validate the Register File behavior.



4.2.5. Verification

All test cases produced the expected register file outputs. The reset test confirmed that the register file returns to its initialized values when **reset** is asserted and a clock pulse is applied. In this implementation, reading **x3** and **x1** after reset produces **rd1=00101011** and **rd2=00101001**.

The write test verified that register **x3** is updated to **10101010** only when **we = 1** and the clock is triggered. The disabled-write test confirmed that when **we = 0**, the register value remains unchanged even if a clock pulse is applied. Finally, the reset-after-write test confirmed that the register file can restore its initialized values after a previous write. The Register File component is validated for integration into the processor datapath.

4.3. Control Unit

The Control Unit decodes the instruction opcode and function fields to generate the processor control signals. It translates **op**, **f3**, and **f7** inputs into branch, memory, ALU, and register control outputs used by the datapath.

4.3.1. Specifications

The figure below shows the Control Unit circuit in CircuitVerse with the input and output signals labeled.

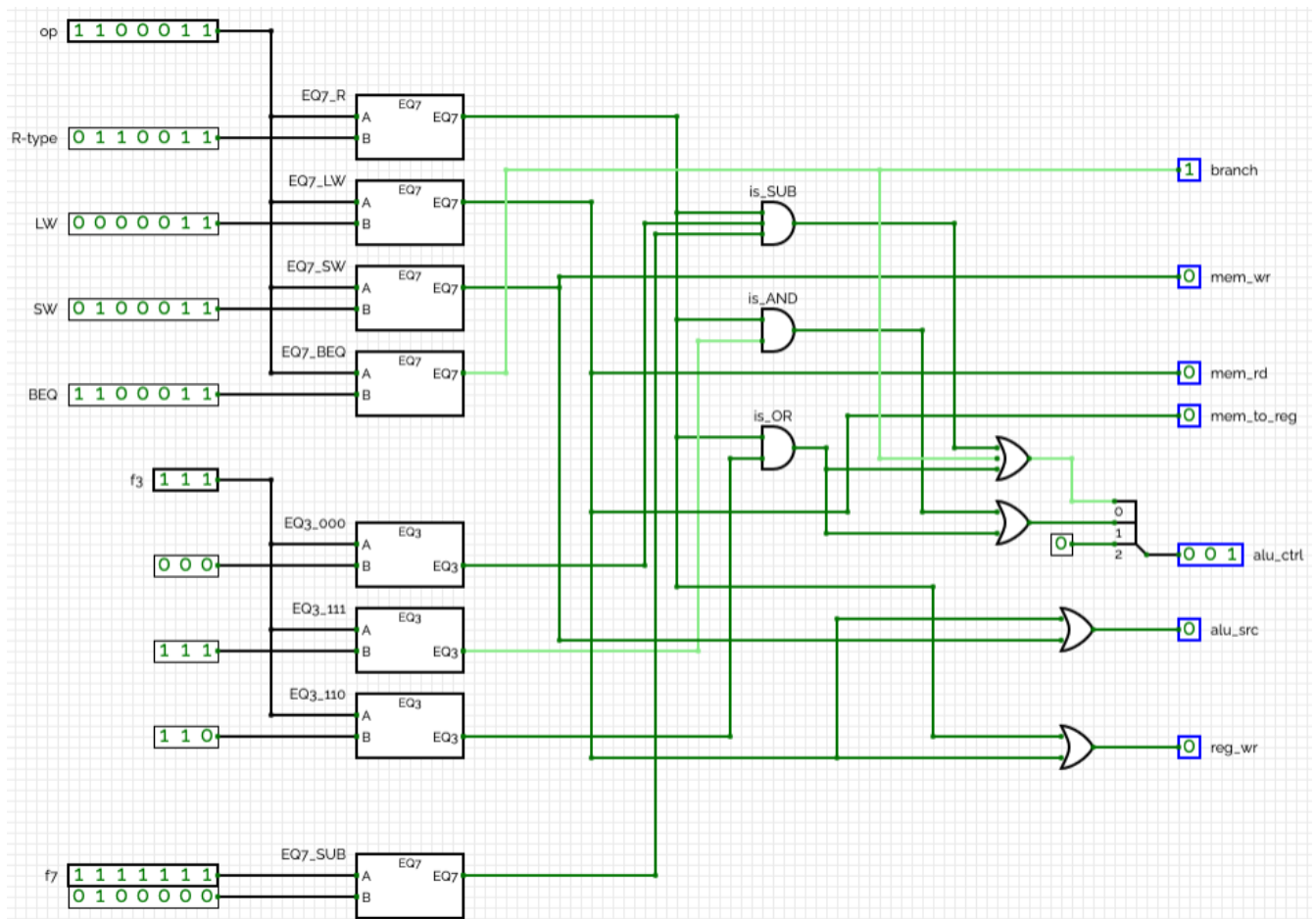


Figure 3. Control Unit Circuit Diagram

- **op** (7-bit): Opcode field from the instruction.

- **f3** (3-bit): funct3 field (function sub-code).
- **f7** (7-bit): funct7 field (function sub-code).
- **branch** (1-bit): Asserted for branch instructions.
- **mem_wr** (1-bit): Memory write enable for store instructions.
- **mem_rd** (1-bit): Memory read enable for load instructions.
- **mem_to_reg** (1-bit): Select memory data for writeback.
- **alu_ctrl** (3-bit): ALU operation code (mapped to ALU control inputs).
- **alu_src** (1-bit): Select immediate as ALU second operand.
- **reg_wr** (1-bit): Register file write enable.

NOTE

For load (**LW**), store (**SW**), and branch-equal (**BEQ**) instruction types, the **f3** and **f7** fields do not affect the control signals in this implementation, so they are treated as don't care.

4.3.2. Testing Methodology

The test suite provides opcode and function-field patterns and verifies the generated control signals. Test cases exercise:

1. **R-type variants**: Multiple **f3/f7** combinations that map to different **alu_ctrl** values and exercise register-write behavior, specifically for the ADD, SUB, AND, and OR instructions in that order.
2. **Load (LW)**: Confirms memory-read path and memory-to-register selection.
3. **Store (SW)**: Confirms memory-write enable and ALU source selection for addressing.
4. **Branch (BEQ)**: Verifies branch output assertion for branch-type opcodes.

Each category uses representative binary encodings from the CSV tests and checks the seven control outputs.

4.3.3. Test Cases and Results

Table 5. R-Type Variants

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
0110011	000	0000000	0	0	0	0	000	0	1
0110011	000	0100000	0	0	0	0	001	0	1
0110011	111	0000000	0	0	0	0	010	0	1
0110011	110	0000000	0	0	0	0	011	0	1

These R-type test cases validate decoding into register-write operations and correct **alu_ctrl** selection for different function fields.

Table 6. Load (LW)

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
0000011	XXX	XXXXXXXX	0	0	1	1	000	1	1

This test case confirms that a load instruction asserts **mem_rd**, selects memory data for writeback, and uses the immediate as the ALU source.

Table 7. Store (SW)

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
0100011	XXX	XXXXXXXX	0	1	0	0	000	1	0

The store test case verifies **mem_wr** assertion and that the ALU uses the immediate for address calculation; no register write is performed.

Table 8. Branch (BEQ)

op	f3	f7	branch	mem_wr	mem_rd	mem_to_reg	alu_ctrl	alu_src	reg_wr
1100011	XXX	XXXXXXXX	1	0	0	0	001	0	0

The branch test case validates that the **branch** control is asserted for a branch-type opcode and that no memory or register writes occur.

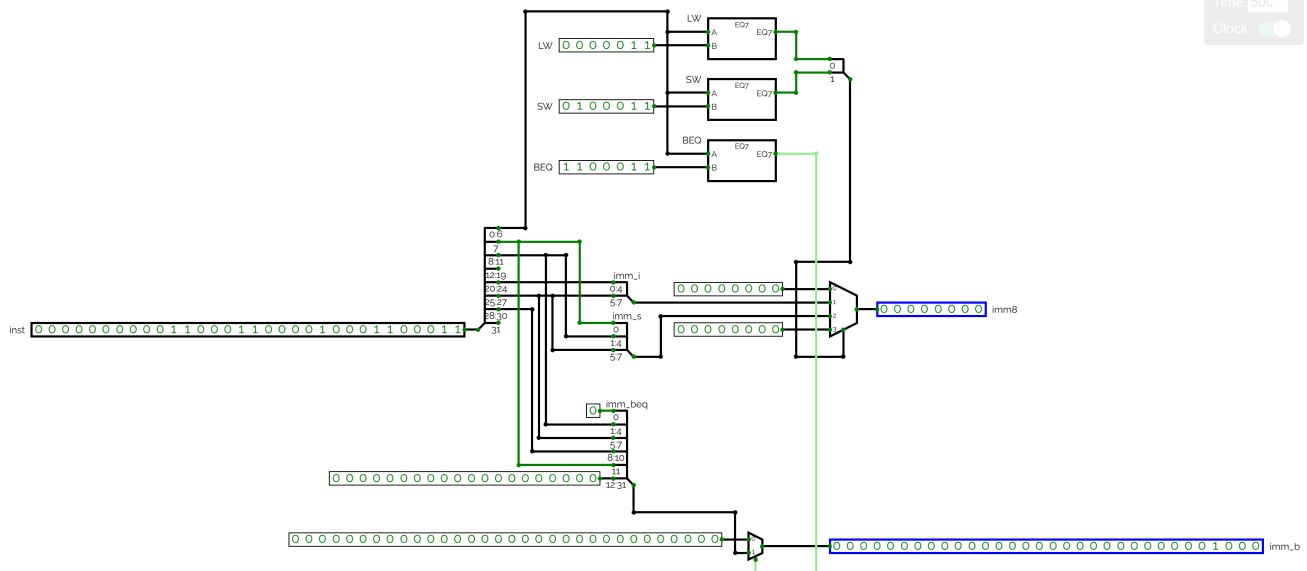
4.3.4. Verification

All test cases produced the expected control outputs. The Control Unit decoding behavior matches the test encodings and the don't-care handling for **f3/f7** in LW, SW, and BEQ cases. The component is validated for integration into the processor datapath.

4.4. Immediate Generator

The Immediate Generator is a combinational component that extracts immediate values from 32-bit RISC-V instructions. The generated immediate is used by instructions such as loads, stores, and branches.

4.4.1. Specifications



The Immediate Generator component has the following input and output signals:

Table 9. Immediate Generator Signals

Signal	Bitwidth	Description
<code>inst</code>	32	Full 32-bit instruction input.
<code>imm8</code>	8	8-bit immediate output used by load and store type instructions.
<code>imm_b</code>	32	Branch immediate output used by branch instructions.

The component identifies the instruction type by checking the opcode bits. Based on the instruction format, it selects the correct immediate field:

Table 10. Immediate Format Behavior

Instruction Type	Example	Immediate Behavior
I-type	<code>lw x1, 0(x0)</code>	Extracts the immediate from bits <code>[31:20]</code> and outputs it through <code>imm8</code> .
S-type	<code>sw x2, 1(x0)</code>	Reconstructs the immediate from bits <code>[31:25]</code> and <code>[11:7]</code> , then outputs it through <code>imm8</code> .
B-type	<code>beq x3, x3, 8</code>	Reconstructs the branch immediate field and outputs it through <code>imm_b</code> .

For the tested branch instruction `beq x3, x3, 8`, the Immediate Generator outputs `00000000000000000000000001000` through `imm_b`, matching the branch offset used by the datapath to compute the correct branch target.

4.4.2. Testing Methodology

The Immediate Generator was tested inside CircuitVerse by manually applying known 32-bit instruction encodings to the `inst` input and observing the generated outputs `imm8` and `imm_b`.

The tests were designed to verify the following behavior:

1. Correct immediate extraction for I-type load instructions.
2. Correct immediate reconstruction for S-type store instructions.
3. Correct branch immediate generation for B-type branch instructions.
4. Correct output behavior for instructions that do not use the immediate generator path.

The complete test cases are documented in [tests/Immediate Generator Tests.csv](#).

4.4.3. Test Cases and Results

Table 11. Immediate Generator Test Cases

Test	Instruction Input	Expected Output	Result
IG_LW_0	0000000000000000010000010000011	imm8=00000000, imm_b=00000000000000000000000000000000 0000	Passed
IG_SW_1	00000000001000000010000010100011	imm8=00000001, imm_b=00000000000000000000000000000000 0000	Passed
IG_BEQ_8	00000000001100011000010001100011	imm8=00000000, imm_b=00000000000000000000000000000000 1000	Passed
IG_ADD	00000000001000001000000100110011	imm8=00000000, imm_b=00000000000000000000000000000000 0000	Passed
IG_NOP	000000000000000000000000000010011	imm8=00000000, imm_b=00000000000000000000000000000000 0000	Passed

4.4.4. Evidence

The following images show the internal CircuitVerse tests used to validate the Immediate Generator behavior.

4.4.5. Verification

All test cases produced the expected immediate outputs. The I-type test confirmed that the Immediate Generator correctly extracts the immediate field for a load instruction. The S-type test confirmed that the component correctly reconstructs the split immediate field used by store instructions.

The B-type test confirmed that the branch immediate is generated correctly for the `beq` instruction. For `beq x3, x3, 8`, the block outputs `00000000000000000000000001000` through `imm_b`, matching the observed CircuitVerse behavior. The non-immediate instruction tests confirmed that R-type operations and `nop` do not incorrectly activate the immediate outputs. The Immediate Generator component is validated for integration into the processor datapath.

4.5. ALU Component

The Arithmetic Logic Unit is a combinational circuit that performs binary arithmetic and logical operations. It serves as the main computational block for data manipulation during instruction execution.

4.5.1. Specifications

The figure below shows the ALU circuit in CircuitVerse with the input and output signals labeled.

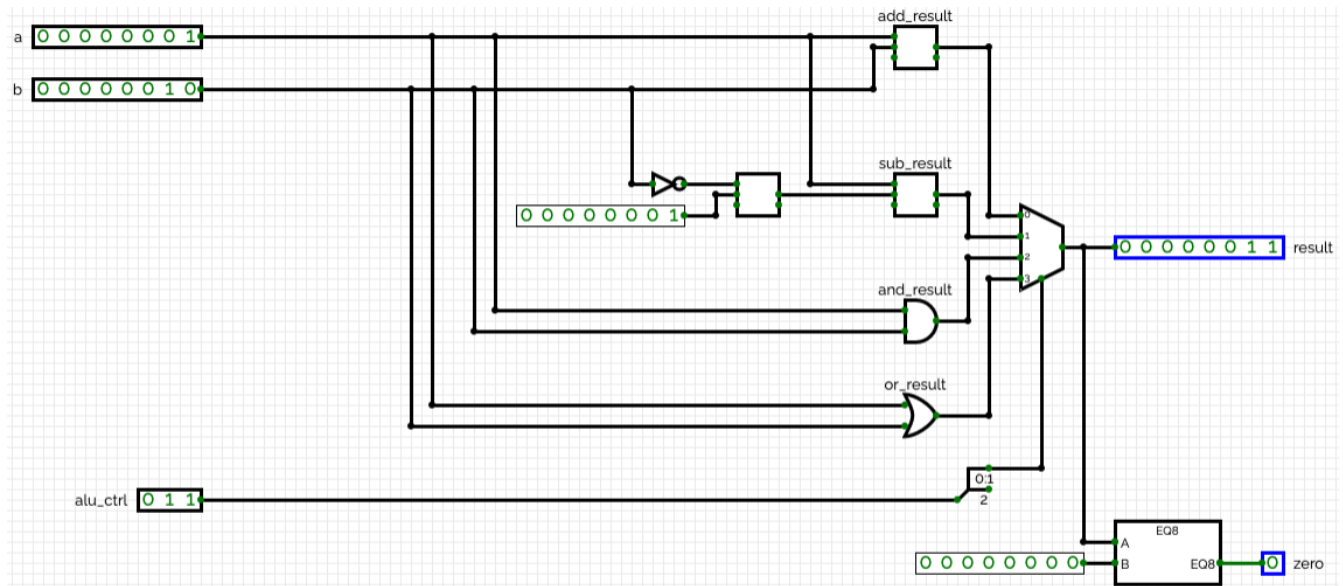


Figure 4. ALU Circuit Diagram

- **a** (8-bit): The first operand for the ALU operation
- **b** (8-bit): The second operand for the ALU operation
- **alu_ctrl** (3-bit): The control signal selecting the ALU operation
- **result** (8-bit): The computed arithmetic or logical result
- **zero** (1-bit): A flag asserted high when the result equals zero

NOTE

The most significant bit of the `alu_ctrl` signal is treated as a don't care bit because

only four operations, representable by 2 bits, are implemented.

The supported operations are defined as follows:

Table 12. ALU Operations

Control Code	Operation	Description
X00	ADD	Binary addition of operands a and b
X01	SUB	Subtraction: a - b using two's complement
X10	AND	Bitwise AND of operands a and b
X11	OR	Bitwise OR of operands a and b

4.5.2. Testing Methodology

The ALU testing protocol exercises all supported operations across diverse operand patterns and edge cases. Six test categories provide coverage:

1. **Addition of Positive Numbers:** Validates binary addition with non-negative operands
2. **Addition of Negative Numbers:** Tests two's complement addition with negative operands
3. **Subtraction of Positive Numbers:** Verifies subtraction including cases producing negative results
4. **Subtraction of Negative Numbers:** Confirms signed subtraction with proper overflow behavior
5. **Bitwise AND Operation:** Validates bit-by-bit conjunction across operand patterns
6. **Bitwise OR Operation:** Confirms bit-by-bit disjunction including complementary patterns

Each category includes three test cases covering normal operation, zero results, edge values, and patterns exercising all bit positions.

4.5.3. Test Cases and Results

Table 13. Addition of Positive Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
00000100 (4)	10000010 (130)	X00	10000110 (134)	0
00000101 (5)	00001001 (9)	X00	00001110 (14)	0
00000000 (0)	00000000 (0)	X00	00000000 (0)	1

Addition of positive operands produces correct sums with the zero flag asserted when both inputs are zero.

Table 14. Addition of Negative Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
11111100 (-4)	11111101 (-3)	X00	11111001 (-7)	0

Input a	Input b	alu_ctrl	Result	Zero Flag
11111111 (-1)	11111111 (-1)	X00	11111110 (-2)	0
10000000 (-128)	00000001 (1)	X00	10000001 (-127)	0

Two's complement negative operands are added correctly, with proper overflow handling within the 8-bit domain.

Table 15. Subtraction of Positive Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
00001010 (10)	00000100 (4)	X01	00000110 (6)	0
00001111 (15)	00001111 (15)	X01	00000000 (0)	1
00000100 (4)	00001010 (10)	X01	11111010 (-6)	0

Subtraction of positive operands generates correct results, including cases where the result is negative.

Table 16. Subtraction of Negative Numbers

Input a	Input b	alu_ctrl	Result	Zero Flag
11111100 (-4)	11111101 (-3)	X01	11111111 (-1)	0
10000000 (-128)	00000001 (1)	X01	01111111 (127)	0
11111111 (-1)	11111111 (-1)	X01	00000000 (0)	1

Signed subtraction using two's complement representation produces correct results including scenarios with sign bit inversion.

Table 17. Bitwise AND Operation

Input a	Input b	alu_ctrl	Result	Zero Flag
11110000	00001111	X10	00000000	1
10101010	01010101	X10	00000000	1
11110011	11001100	X10	11000000	0

Bitwise AND correctly implements conjunction logic. Complementary bit patterns produce zero, while overlapping patterns generate appropriate non-zero results.

Table 18. Bitwise OR Operation

Input a	Input b	alu_ctrl	Result	Zero Flag
11110000	00001111	X11	11111111	0
10101010	01010101	X11	11111111	0
00000001	00000010	X11	00000011	0

Bitwise OR correctly implements disjunction logic. Complementary patterns combine to produce all-ones results, and adjacent single-bit patterns merge correctly.

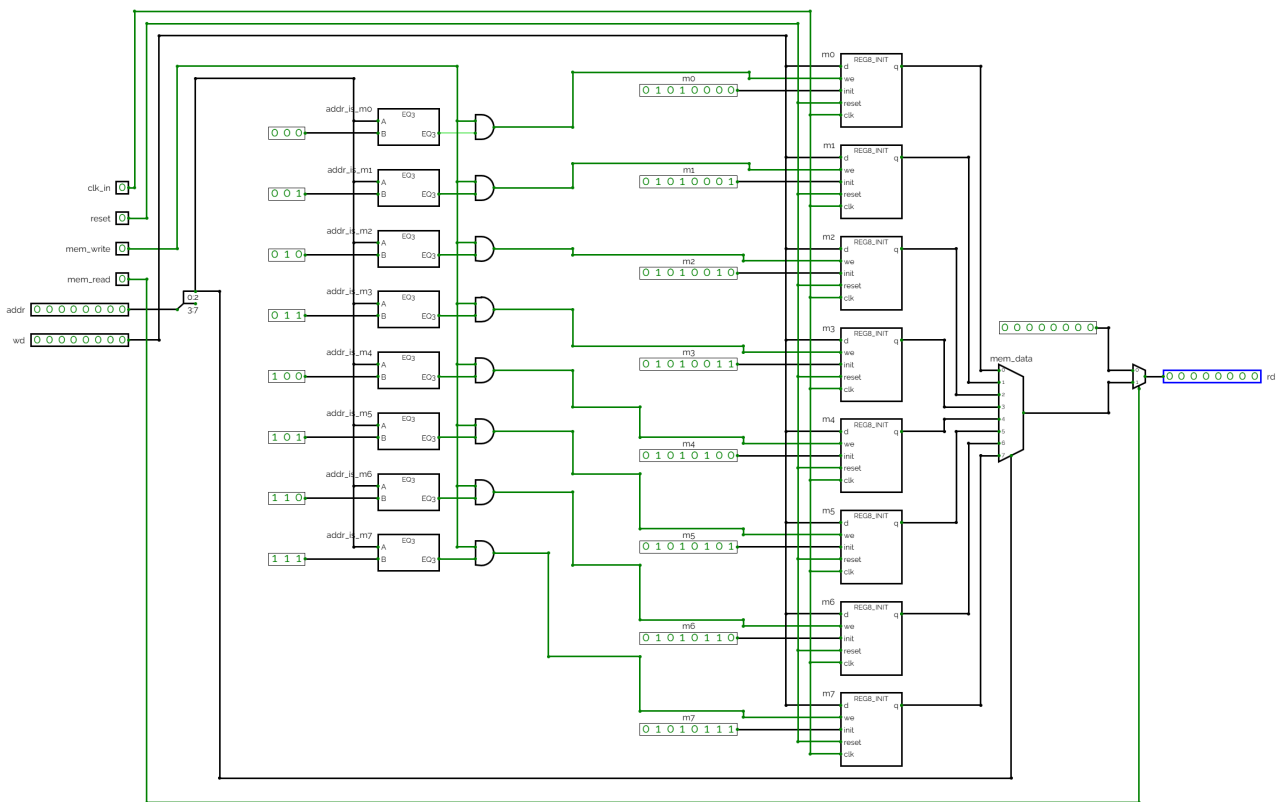
4.5.4. Verification

All ALU test cases execute successfully with results matching predicted values. The ALU correctly implements addition, subtraction, AND, and OR operations with proper two's complement handling and accurate zero flag generation. The component is validated for integration into the processor datapath.

4.6. Data Memory

The Data Memory is a sequential component used to store and retrieve data values during load and store instructions. In this simplified processor, the data memory contains eight 8-bit memory locations.

4.6.1. Specifications



The Data Memory component has the following input and output signals:

Table 19. Data Memory Signals

Signal	Bitwidth	Description
clk_in	1	Clock input. Memory writes and reset behavior occur synchronously when the clock is triggered.
reset	1	Synchronous reset signal. When enabled and the clock is triggered, the memory returns to its initialized values.
mem_write	1	Memory write enable signal. When <code>mem_write = 1</code> , the value on <code>wr</code> is stored at the selected address on a clock pulse.

Signal	Bitwidth	Description
mem_read	1	Memory read enable signal. When <code>mem_read = 1</code> , the selected memory value appears at the output <code>rd</code> .
addr	8	Memory address input used to select the memory location.
wd	8	Write data input. This value is written into memory when <code>mem_write = 1</code> .
rd	8	Read data output.

The memory read output depends on `mem_read`. When `mem_read = 1`, the memory outputs the selected value. When `mem_read = 0`, the output is disabled and returns zero. Writes occur only when `mem_write = 1` and a clock pulse is applied.

4.6.2. Testing Methodology

The Data Memory was tested inside CircuitVerse by manually applying address, write data, read enable, write enable, reset, and clock values.

The tests were designed to verify the following behavior:

1. The memory responds correctly to reset.
2. The memory outputs initialized values when read.
3. A memory location can be written when `mem_write = 1`.
4. The written memory value can be read back correctly.
5. A memory location is not modified when `mem_write = 0`.
6. The read output is disabled when `mem_read = 0`.
7. The memory returns to its initialized values after reset.

The complete test cases are documented in `tests/Data Memory Tests.csv`.

4.6.3. Test Cases and Results

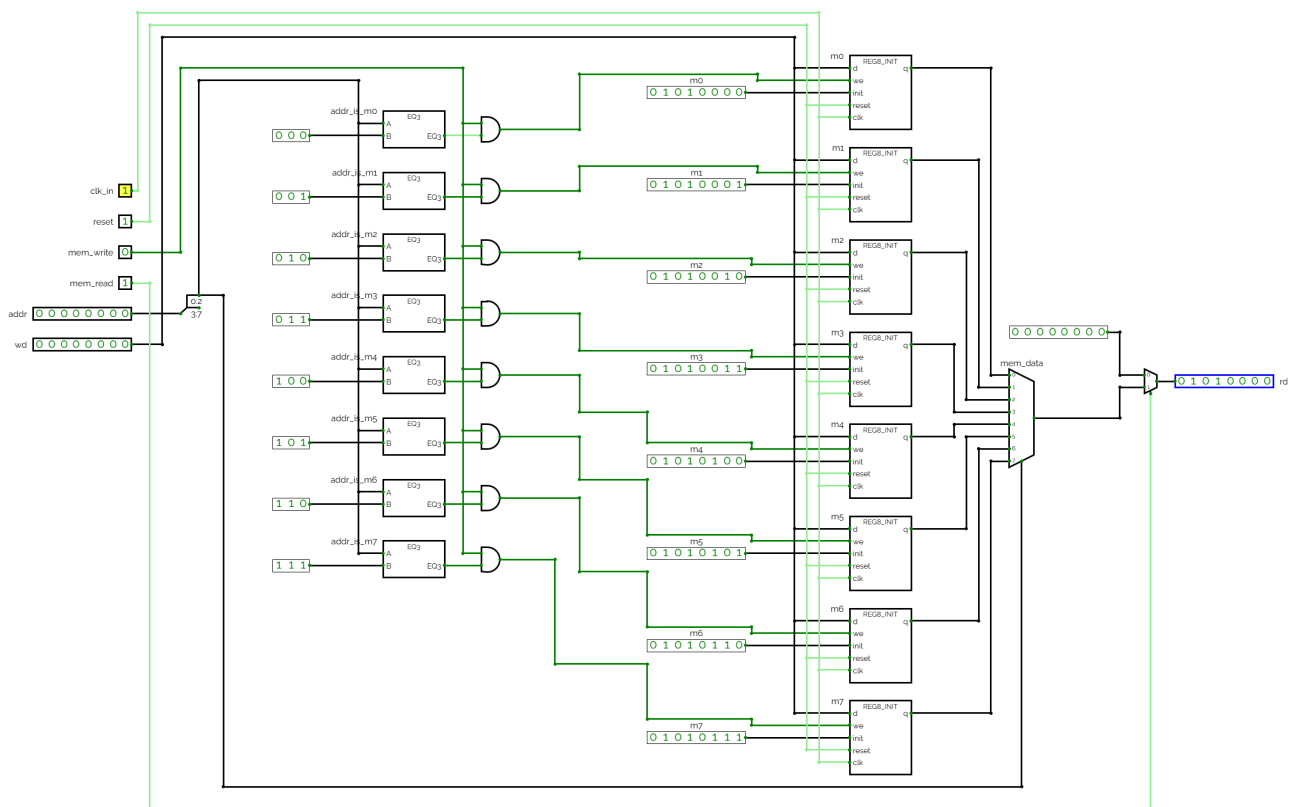
Table 20. Data Memory Test Cases

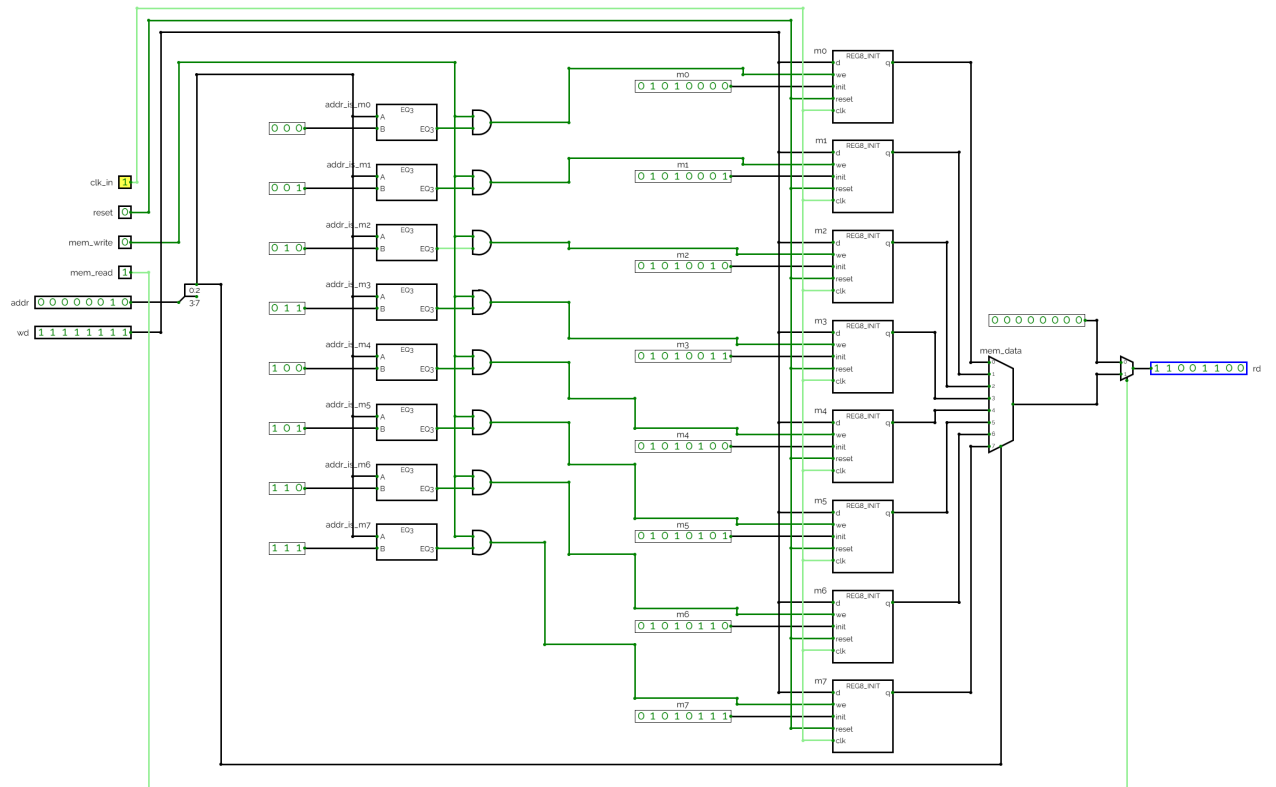
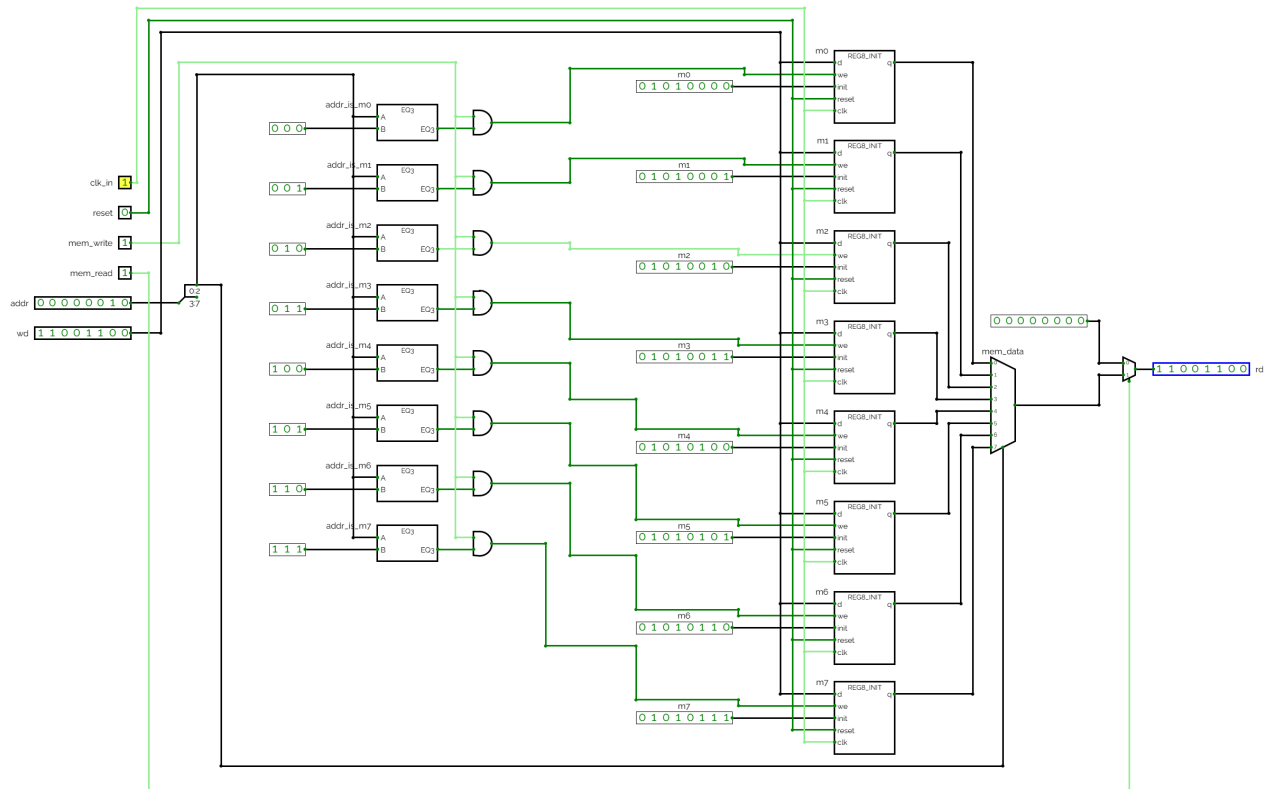
Test	Inputs Applied	Expected Output	Result
DM_RESET_READ_M0	<code>clk_in=1, reset=1, mem_write=0, mem_read=1, addr=00000000, wd=00000000</code>	<code>rd=01010000</code>	Passed
DM_READ_M1	<code>clk_in=0, reset=0, mem_write=0, mem_read=1, addr=00000001, wd=00000000</code>	<code>rd=01010001</code>	Passed
DM_READ_M7	<code>clk_in=0, reset=0, mem_write=0, mem_read=1, addr=00000111, wd=00000000</code>	<code>rd=01010111</code>	Passed

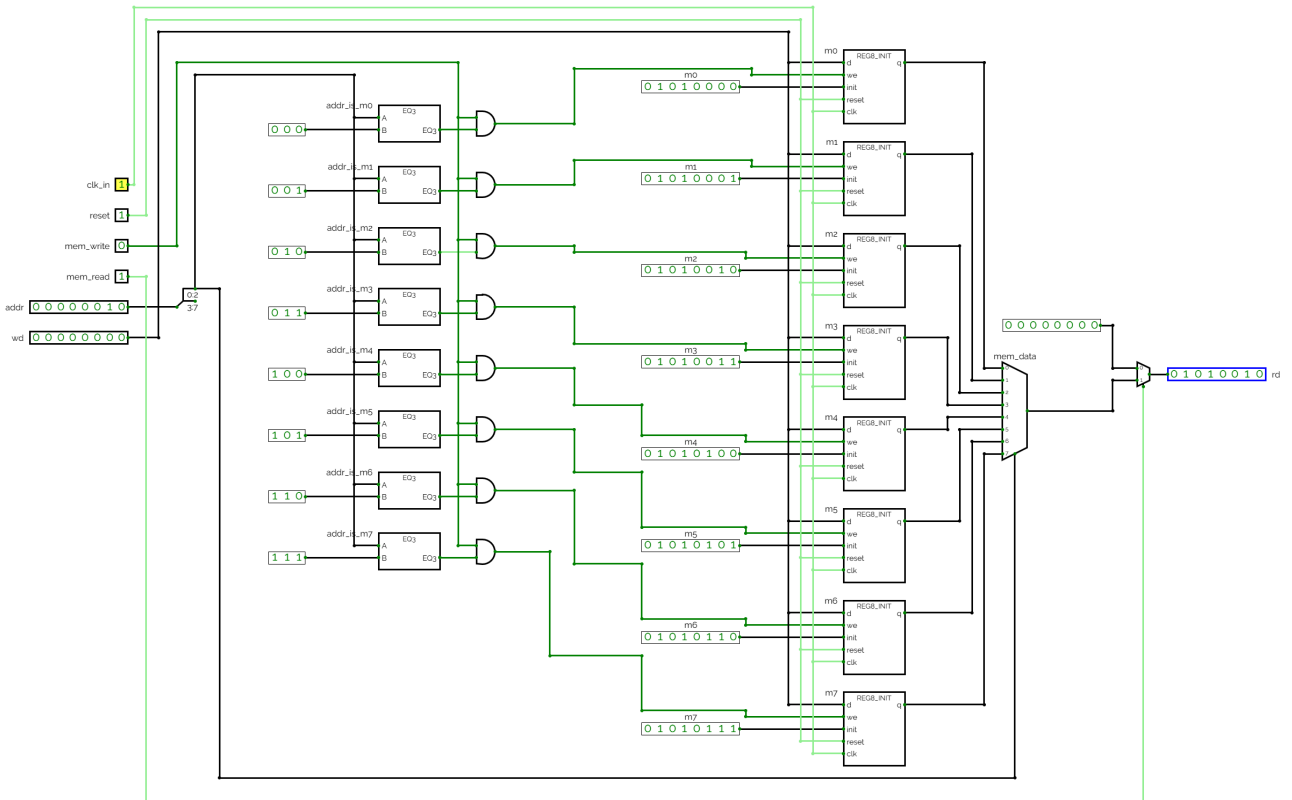
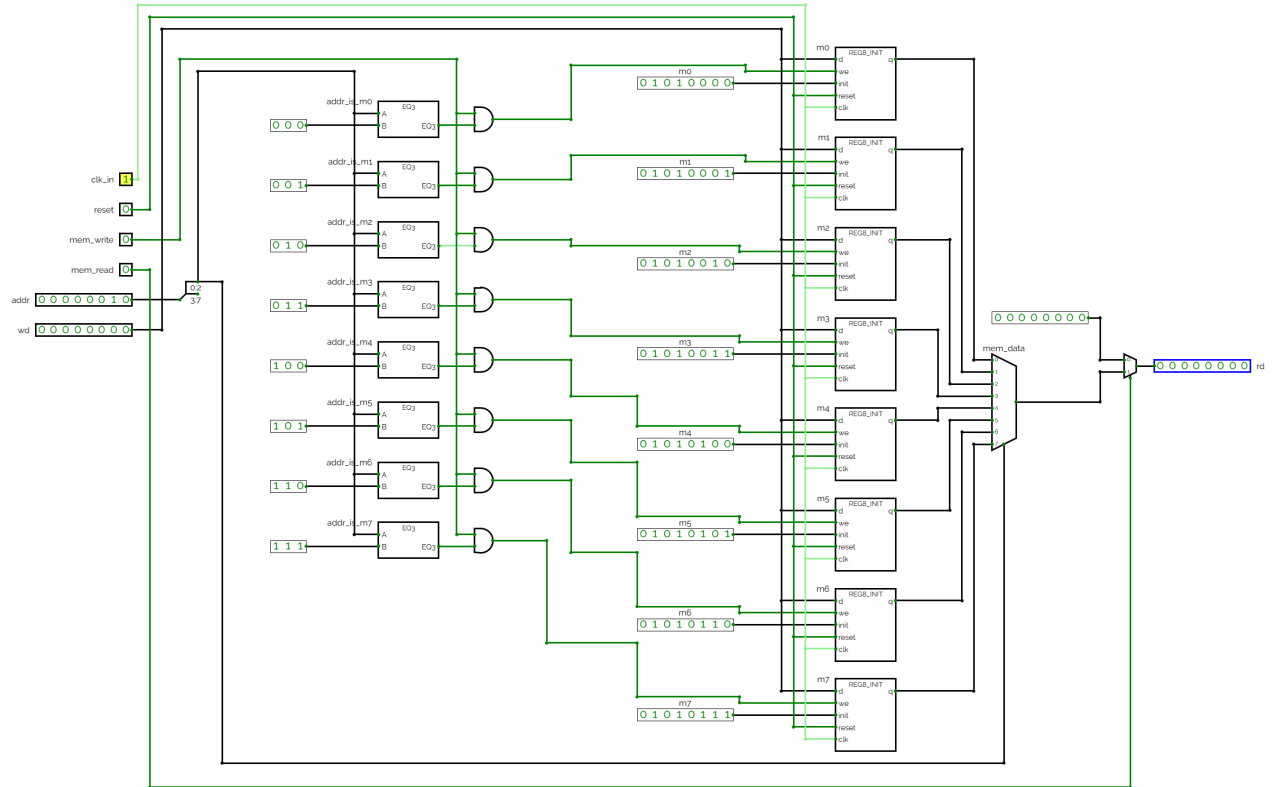
Test	Inputs Applied	Expected Output	Result
DM_WRITE _M2_CC	clk_in=1, reset=0, mem_write=1, mem_read=1, addr=00000010, wd=11001100	rd=11001100	Passed
DM_BLOCK _WRITE_M 2	clk_in=1, reset=0, mem_write=0, mem_read=1, addr=00000010, wd=11111111	rd=11001100	Passed
DM_READ_ DISABLED	clk_in=0, reset=0, mem_write=0, mem_read=0, addr=00000010, wd=00000000	rd=00000000	Passed
DM_RESET _AFTER_W RITES_M2	clk_in=1, reset=1, mem_write=0, mem_read=1, addr=00000010, wd=00000000	rd=01010010	Passed

4.6.4. Evidence

The following images show the internal CircuitVerse tests used to validate the Data Memory behavior.







4.6.5. Verification

All test cases produced the expected data memory outputs. The reset test confirmed that the memory returns to its initialized values when **reset** is asserted and a clock pulse is applied. The read tests confirmed that the output **rd** correctly reflects the value stored at the selected address when **mem_read** = 1.

The write test verified that memory address `00000010` is updated to `11001100` only when `mem_write = 1` and the clock is triggered. The disabled-write test confirmed that memory contents remain unchanged when `mem_write = 0`, since address `00000010` remained equal to `11001100` even when `wd = 11111111`. The disabled-read test confirmed that the output is cleared to `00000000` when `mem_read = 0`. Finally, the reset-after-writes test confirmed that memory address `00000010` returns to its initialized value `01010010` after reset. The Data Memory component is validated for integration into the processor datapath.

5. Part 3: Processor Integration and Validation

After verifying the individual components, the complete processor is validated by executing the full instruction sequence. This integration phase confirms that the components interact correctly and that the processor achieves the intended behavior when executing the program.

5.1. Testing Protocol

Full processor validation is performed using the CircuitVerse simulation environment shown below:

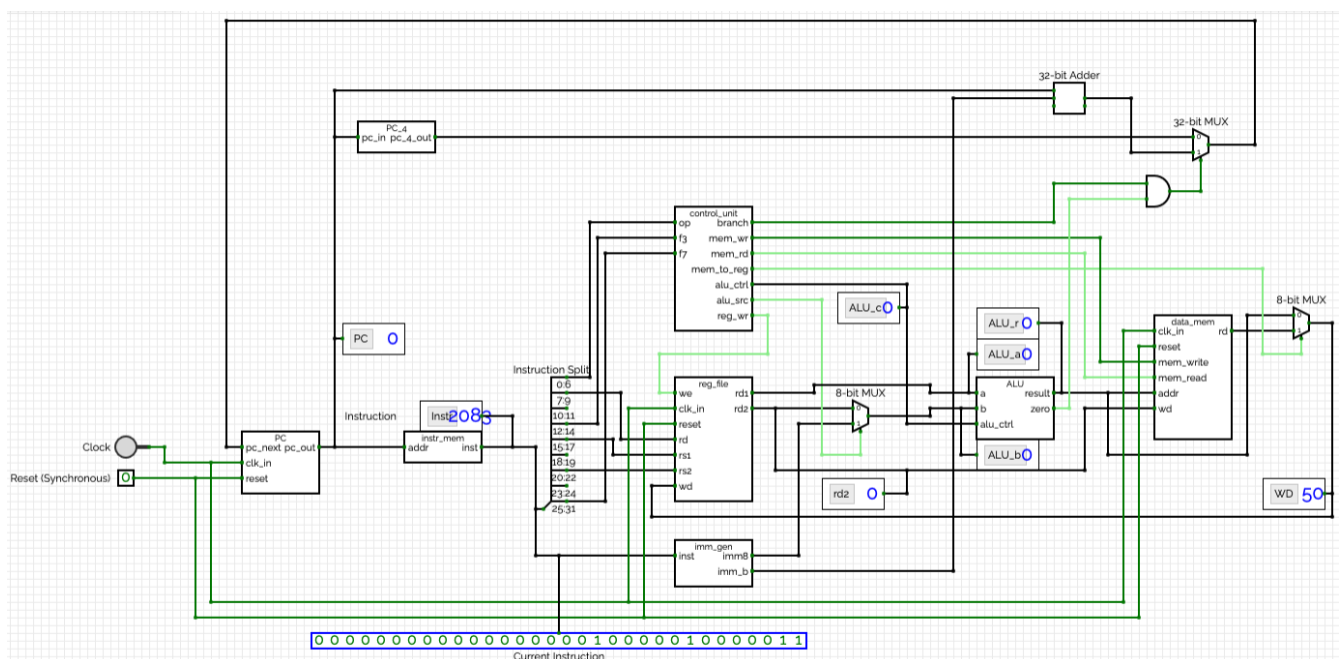


Figure 5. Full Processor Simulation in CircuitVerse

The steps are as follows:

1. Set the reset signal to (1), and force a single clock cycle to initialize the processor state.
2. Set the reset signal back to (0) to allow normal operation.
3. Force clock cycles one by one and observe the processor executing the instruction sequence defined in the instruction memory from the timing diagram view, while monitoring key signals and outputs at each clock cycle.
4. Compare the observed behavior against expected results based on the instruction sequence and

initial processor state.

For context, the contents of the instruction memory, register file, and data memory at the start of execution are shown below:

Table 21. Instruction Memory

Address	Instruction (Hex)	Instruction (Assembly)
0x00	0x00002083	lw x1, 0(x0)
0x04	0x00208133	add x2, x1, x2
0x08	0x002020A3	sw x2, 1(x0)
0x0C	0x00318463	beq x3, x3, 8
0x10	0x40638233	sub x4, x7, x6
0x14	0x0020E2B3	or x5, x1, x2
0x18	0x00000013	addi x0, x0, 0 (nop)
0x1C	0x00000013	addi x0, x0, 0 (nop)

Table 22. Register File

Register	Initial Value (Hex)
x1	0x00000029
x2	0x0000002A
x3	0x0000002B
x4	0x0000002C
x5	0x0000002D
x6	0x0000002E
x7	0x0000002F

Table 23. Data Memory

Address	Initial Value (Hex)
0x00	0x00000050
0x01	0x00000051
0x02	0x00000052
0x03	0x00000053
0x04	0x00000054
0x05	0x00000055
0x06	0x00000056
0x07	0x00000057

NOTE Hexadecimal values are used for clarity because the CircuitVerse simulation

displays values in hex in the timing diagram and its corresponding "Flag" elements.

5.2. Signal Monitoring Points

The following signals are sufficient to verify correct execution of the given instruction sequence:

- Program Counter (**PC**): Current instruction address
- Current Instruction (**Instr**): Instruction fetched from instruction memory
- Register x1 (**x1**): Destination of the **lw** instruction
- Register x2 (**x2**): Destination of the **add** instruction
- Data Memory Address 0x01 (**mem[0x01]**): Destination of the **sw** instruction
- Register x4 (**x4**): Destination of the skipped **sub** instruction
- Register x5 (**x5**): Destination of the **or** instruction executed after the branch

Monitoring these values allows verification of instruction fetch, arithmetic execution, memory access, branch behavior, the absence of writes from the skipped **sub** instruction, and correct execution of the subsequent **or** instruction.

5.3. Instruction Execution Verification

The expected state transitions for the monitored values are shown below.

Table 24. Expected Results

PC	Instr	x1	x2	mem[0x01]	x4	x5
0x00	lw x1, 0(x0)	0x00000050	0x0000002A	0x00000051	0x0000002C	0x0000002D
0x04	add x2, x1, x2	0x00000050	0x0000007A	0x00000051	0x0000002C	0x0000002D
0x08	sw x2, 1(x0)	0x00000050	0x0000007A	0x0000007A	0x0000002C	0x0000002D
0x0C	beq x3, x3, 8	0x00000050	0x0000007A	0x0000007A	0x0000002C	0x0000002D
0x14	or x5, x1, x2	0x00000050	0x0000007A	0x0000007A	0x0000002C	0x0000007A
0x18	nop	0x00000050	0x0000007A	0x0000007A	0x0000002C	0x0000007A
0x1C	nop	0x00000050	0x0000007A	0x0000007A	0x0000002C	0x0000007A

5.4. Processor Validation Results

After executing the instruction sequence and monitoring the key signals, the following timing diagram is obtained, showing the observed values for the monitored signals at each clock cycle:

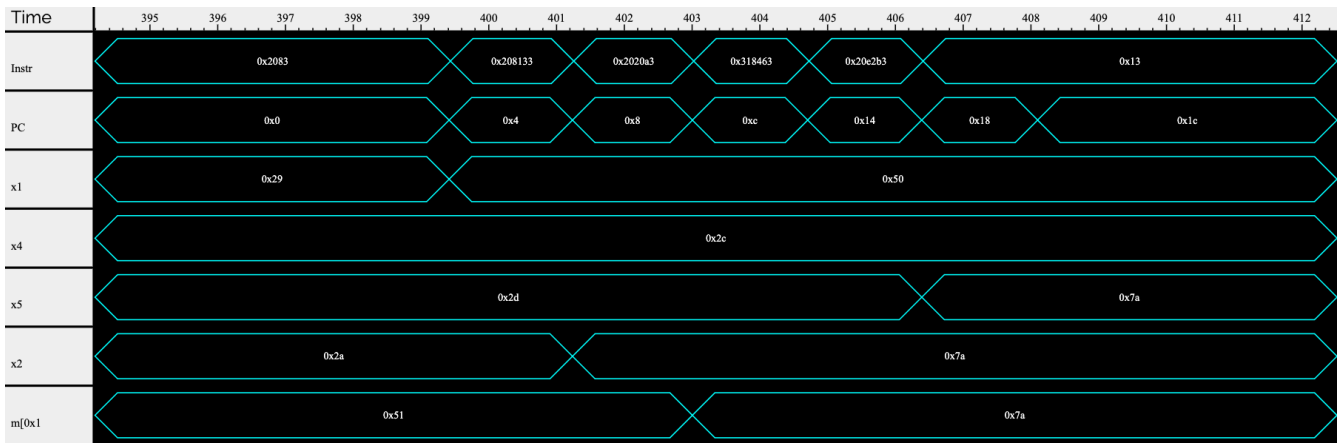


Figure 6. Timing Diagram of Full Processor Execution

NOTE

The shift of 1 cycle of data state between our table and the timing diagram is due to the fact that data is written at the end of the clock cycle, so the observed values for x1, x2, mem[0x01], x4, and x5 reflect the updated state after the instruction execution for that cycle.

The observed values obtained from the timing diagram above are compared against the expected values below:

Table 25. Validation Summary

Verification Item	Expected Result	Observed Result	Status
Final value of x1	0x00000050	0x00000050	Pass
Final value of x2	0x0000007A	0x0000007A	Pass
Final value of mem[0x01]	0x0000007A	0x0000007A	Pass
Final value of x4	0x0000002C	0x0000002C	Pass
Final value of x5	0x0000007A	0x0000007A	Pass
PC value after 0x0C (beq)	0x14	0x14	Pass

Agreement between the observed values and the expected values confirms correct execution of the instruction sequence and successful integration of all processor components.

6. Conclusion

Component-level testing confirmed the correct operation of individual subsystems, while integration testing verified proper interaction within the complete processor datapath.

Testing the components in isolation before system-wide integration ensures that the processor architecture is sound and that each element functions as designed. The test protocols provide evidence of functional correctness and support further use of the processor design.

All major processor subsystems, as well as the entire processor itself, have been verified to meet their functional specifications. The documented test results provide a complete record of validation activities and demonstrate readiness for operation with the full instruction set.

The full project history, including the testing and validation work, is available in [our GitHub repository](#) for reference.